Aarush Singh

647-723-5484 | | a882sing@uwaterloo.ca | LinkedIn | Github

Education

University of Waterloo — Waterloo, ON
Bachelor of Mathematics in Computer Science (Honours) – Presidential Scholarship — *Sep. 2025 – Sep. 2030*

Wilfrid Laurier University — Waterloo, ON
Bachelor of Business Administration (Honours) — *Sep. 2025 – Sep. 2030*

Technical Skills

Languages: C++, C, Python, SQL, JavaScript, TypeScript
Frameworks & Libraries: Next.js, React, Tailwind CSS, pandas, NumPy, Matplotlib, Pytest, Flask
Cloud, Databases & Tools: AWS (Lambda, EventBridge, S3, Aurora, Bedrock), PostgreSQL, Supabase, Git , Bash, Linux, Docker, Sharepoint
Certifications: Bloomberg Market Concepts (BMC), Bloomberg Finance Fundamentals

Experience

Research Analyst – FICC — Waterloo, ON
Whitestone Capital — *Oct. 2026 – Present*
- Conduct fundamental research on **rates, currencies, and commodities** for a cross-university equity research firm.
- Develop thesis-driven, investment-grade research through industry analysis and **financial modeling** across FICC markets.

Software Engineering Intern — New Delhi, IN
Ministry of Statistics and Programme Implementation — *May 2026 – Aug 2026*
- Built and maintained databases with SQL, Python, and Excel that now hold more than **10.8 million records**.
- Designed Python ETL pipelines loading **80+ survey workbooks** (Irrigation, ASUSE, Gender) into the Unified Data Platform.
- Contributed to this flagship Government of India platform, which publishes more than **100,000 datasets**.
- Benchmarked AI data processing tools on three flagship national datasets (NSS, NAS, AISHE) to improve reliability.

Projects

Campus Marketplace Web Application | *Next.js 15, Supabase (PostgreSQL, Auth, Storage), Tailwind CSS* — 2026
- Built a full-stack marketplace for **@uwaterloo.ca** accounts with listings, wanted posts, filters, and sortable search.
- Built sign-up and login with Supabase Auth, including **email confirmation**, **admin roles**, and **protected routes**.
- Designed **11 PostgreSQL tables** with **53 row-level security policies** and **27 indexes** for role-based access.
- Added **multi-image uploads** to Supabase Storage that clean up old images when a listing is edited or deleted.
- Built an **admin dashboard** for content moderation and onboarded **50+ early-adopter students** to the platform.

Indian Job Market Visualiser | *Python, pandas, JavaScript, OpenRouter* — 2026
- Adapted Andrej Karpathy's U.S. occupation treemap for India, grouping **1.15M PLFS** survey records into 126 occupations.
- Matched India's official 57.4% worker-population ratio and scaled estimates to **617M workers** using official projections.
- Rebuilt the front end as a dependency-free canvas treemap with a **squarified layout** and hover tooltips, written from scratch.
- Scored all 126 occupation groups for AI exposure with Gemini 3.6 Flash and documented each assumption for auditing.

Brokeberg: Political Intelligence Terminal | *Python, AWS, PostgreSQL/pgvector, Bedrock, Next.js* — 2026
- Built an **event-driven ETL pipeline** on **AWS Lambda, EventBridge, S3, and Aurora** ingesting **7 public data sources**.
- Modeled the data as a **knowledge graph** that tracks each fact's source and resolves **100+ entities** to canonical IDs.
- Built a **6-pass LLM extraction pipeline** on **Bedrock** that ties each claim to source text, using Haiku or Sonnet per pass.
- Built an **evaluation harness** scoring each pass's precision and recall on **43 labeled items** before every prompt change.
- Clustered events with **pgvector** and linked economic data to Senate races in a Bloomberg-style **Next.js terminal**.

Forest Analyzer | *Python, JavaScript, HTML/CSS, Flask, Google Earth Engine, Leaflet* — 2026
- Built a web app that measures forest loss in any region worldwide using Sentinel-2 satellite imagery at **10m resolution**.
- Estimated vegetation loss and gain with **NDVI** by comparing satellite images of the same area year over year.
- Used the **Google Earth Engine API** to process imagery and **Leaflet** to show color-coded loss and gain on a live map.

APEX: Multi-Agent Motorsport Safety Stress Tester | *Python, NumPy, FastAPI, SQLite, Next.js* — 2026
- Built a deterministic, vectorized racing simulator running **22 driver agents** at 20 Hz across **10,000+ scenarios** per batch.
- Scored each scenario with standard safety measures (**TTC, PET, SSAM-style conflicts**) to find recurring risky interactions.